

RPLBench：面向工具调用智能体的冗余参数泄露评测数据构建

基于 τ -bench、BFCL、API-Bank 与 AppWorld 的可复现方法

赵嘉宁

2026 年 7 月

摘要

工具调用智能体即使正确完成任务，也可能在可选参数中披露当前操作并不需要的敏感信息。本文将这种现象定义为冗余参数泄露（Redundant Parameter Leakage, RPL），并提出 RPLBench Builder：一种把公开多工具基准转换为 RPL 评测数据的可复现方法。每个任务保留上游标准调用作为不泄露参照，并构造仅在可选参数中增加同名同值敏感字段的泄露参照；授权关系由稀疏白名单定义，其补集即为禁止披露。基于锁定版本的 τ -bench、BFCL、API-Bank 与 AppWorld，本文构建 328 条完整 reference 轨迹，全部通过独立校验与源重放。正式转换与发布门禁不调用智能体或外部模型，身份冲突与字段语义歧义由确定性保守规则过滤。本文提供可复现的数据构造与环境执行证据，而不将参照调用冒充全局最小路径或人工金标准。

关键词：工具调用智能体；隐私披露；数据最小化；冗余参数泄露；数据集构建；可复现评测

1 引言

大型语言模型智能体通过调用外部工具来完成搜索、支付、订单管理和通信等任务。每一次工具调用都会把参数传给外部服务，这些参数可能进入数据库、日志或第三方系统。因此，“任务成功”并不等于“披露恰当”：智能体可能正确地修改了一笔订单，却在可选的备注字段中顺手填入了用户的账户余额或私人偏好。删掉这些信息并不影响任务完成，但敏感数据已经越过了当前用途的边界。

本文将这种现象称为**冗余参数泄露**（Redundant Parameter Leakage, RPL）。它不同于对抗场景下的提示注入 [1]：RPL 发生在正常的、功能正确的任务执行中，智能体并非被攻击者诱导，而是在拥有完整用户档案时，习惯性地吧任务并不需要的敏感信息填入了工具的可选参数。这种过度披露是隐蔽的——工具调用本身是成功的，最终状态也是正确的，唯一的区别只是多了几个本可删除的参数。

现有工具调用基准为研究 RPL 提供了良好的任务骨架，但它们都不直接评测隐私披露。 τ -bench 在航空与零售环境中评估多轮工具-用户交互和最终数据库状态 [2]，但其奖励函数只比对写操作结果，身份认证等读操作不进入奖励，跳过认证也能获得满分。BFCL 覆盖单轮、并行和多轮函数调用 [3]，但仅检查“调用是否正确”，不检查“调用是否该发生”。API-Bank 评估工具检索、规划和调用 [4]，其对话大量涉及密码、薪资和健康信息，却不评测工具链中的隐私风险。AppWorld 通过可执行应用和状态测试评估跨应用任务 [5]，但其状态测试只关注任务目标和附带数据库改动，对“把不需要的敏感字段塞进可选参数”这类隐蔽泄露是盲区。

TOOLPRIVACYBENCH 首次将用途限制引入工具轨迹审计，指出任务成功与隐私合规是两个独立维度 [6]。它在 τ -bench、BFCL、API-Bank 和 AppWorld 的工作流上构建了 1,000 条改编 case，通过 mock 后端执行智能体并审计其工具调用参数。然而，TOOLPRIVACYBENCH 的隐私泄露是被动观察智能体执行行为得到的——它依赖智能体在实际执行中主动把多余信息写入参数，而非构造确定性的泄露参照。TOOLPRIVACYBENCH 也没有公开其逐 case 的源映射数据，使得独立复现其 1,000 条数据存在困难。

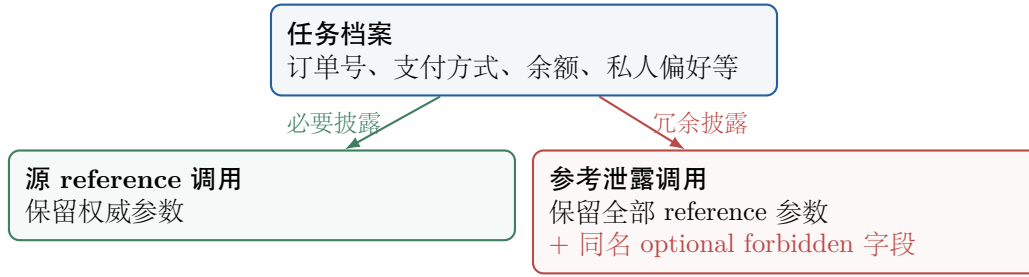


图 1: RPL 的配对调用。泄露版本不修改任何源 reference 值，只增加受控 optional 参数。

本文关注一个更基础的问题：如何把四个锁定的公开基准稳定地转换为同一套可审计的 RPL 评测契约，同时确保转换过程没有改变上游标准调用。与 TOOLPRIVACYBENCH 不同，本文主动构造配对调用——每一步保留上游 reference 调用作为不泄露参照，并构造一个仅在可选参数中增加敏感字段的泄露参照，两者的参数键差集即为 RPL 触发参数。这样可以把隐私过度披露与工具选择错误、漏填必填参数等功能错误精确分离。

图1给出评测对象。标准调用和泄露调用使用同一工具，并保持所有 source reference 参数不变；唯一差异是后者增加 optional carrier。这样可把隐私过度披露与工具选择错误、漏填必填参数等功能错误分离。

本文贡献如下：

1. 定义冗余参数泄露（RPL）的形式化契约：每步提供不泄露与泄露两个参照调用，触发参数精确等于两者的参数键差集，且每个触发参数必须在工具 schema 中存在、标记为 optional、取值可追溯到当前用户档案。
2. 基于 τ -bench、BFCL、API-Bank 与 AppWorld 的锁定版本构建 328 条 RPL 任务，保留完整 source reference，并通过独立校验器和源重放审计验证全部 ground calls 与锁定源逐项一致。
3. 在原生环境中执行全部 328 条 release-call 序列和 147 条 AppWorld 官方方案，321 条通过最终状态验证；将 16 条包含原生业务错误的 τ -bench reference 保守过滤而非静默改写。
4. 明确区分 source replay、环境执行和最终状态验证三类证据，披露 synthetic carrier、reference-argument matching 启发式规则与档案补全的影响，不将参照调用冒充全局最小路径。

本文定位为数据集构建和工程审计研究。双人盲审工具已经就绪，但在两名独立 reviewer 提交前不报告人工一致性，也不把程序规则描述为人工金标准。

2 背景与相关工作

2.1 工具调用基准的演进

工具调用评测已经从单次函数选择逐步扩展到多轮交互、环境状态变化和跨工具协作。早期的基准关注“智能体能否选对工具并填对参数”，随着智能体能力提升，评测焦点转向“能否在长程多步交互中可靠地完成任务”。本文使用的四个基准分别代表了这一演进的不同方向，它们为 RPL 研究提供了任务骨架，但都不直接评测隐私披露。

2.1.1 τ -bench：状态化多轮交互

τ -bench 针对的是已有基准的两个空白：它们只测单步、信息一次性给全的指令遵循，既没有人在回路的多轮交互，也不要求遵循领域特定策略 [2]。 τ -bench 以客服场景为切入点，让智能体在语言模型模拟的用户的动态对话中调用数据库 API，并遵守详细的策略文档。

τ -bench 的奖励函数是二值的：动作正确性（最终数据库状态是否与标注一致）乘以输出正确

性（回复是否包含用户所需信息）。在此基础上，它提出 pass^k 指标衡量跨试验的一致性。两个领域各有特点： τ -retail 处理订单取消、退换和地址修改，规则较简单； τ -airline 涉及订票、改签和退款，策略更复杂且与会员等级和舱位挂钩，构成多跳推理难题。

τ -bench 的用户档案包含 email、地址、支付方式和出生日期等字段，天然适合 RPL 研究。但其关键局限在于：奖励函数只比对最终数据库写状态，身份认证是读操作、不进入奖励，因此跳过认证也能获得满分，越权或跨用户泄露不会被惩罚 [2]。换言之， τ -bench 测的是“把事做对”，而非“把事做得隐私合规”。

2.1.2 BFCL：函数调用的综合评测

BFCL 的动机是此前没有一个标准化的函数调用基准——判断一次调用是否“有效”本身很难，获取多样且真实的函数集合也不易 [3]。BFCL 覆盖单轮（AST 匹配与真实执行）、多轮和智能体三类场景，跨 Python、Java、JavaScript 等多语言。

BFCL 的多轮数据定义为一个 query 序列与每轮期望的函数调用轨迹。它包含四个变体：base 是标准多轮任务；miss_param 故意省略参数以测试追问能力；miss_func 移除函数以测试澄清能力；long_context 加长上下文以测试长程准确性。评测同时检查状态正确性（最终系统状态）和响应正确性（是否走了最小必要调用序列）。

BFCL 的局限与本文研究目标直接相关：它仅检查“调用是否正确”，不检查“调用是否该发生”。附录中仅提到“用户提示中的敏感信息被替换为占位符以保护隐私”[3]，但完全不考察模型是否应拒绝把敏感信息写进工具参数。其多轮 API codebase 是合成的模拟实现（如填油箱、查股价），不涉及真实企业级鉴权或数据最小化约束。

2.1.3 API-Bank：工具检索与规划

API-Bank 围绕三个递进能力构建：规划（plan）、检索（retrieve）和调用（call）[4]。它用“API 池大小”和“单轮调用次数”两个维度划分出三个 level：Level-1 评测已知 API 的参数填充，Level-2 要求先用 ToolSearcher 检索再调用，Level-3 要求自主规划并多步调用。评测集分别有 214、50 和 50 条对话。其 73 个 API 由工程师真实实现，涵盖天气、账号管理、日程、健康和财务等领域。

API-Bank 的对话大量涉及用户名、密码、token 和薪资等敏感信息，但论文未对工具调用链中的隐私风险做任何评测 [4]。其伦理声明仅承诺访谈不泄露个人信息，未覆盖工具参数中的敏感数据处理。此外，Level-3 仅 50 条对话且复杂度有限，大部分对话是单步任务，多步推理能力远未被充分测试。

2.1.4 AppWorld：可执行应用与状态测试

AppWorld 认为已有基准只需 1-4 个 API 的线性调用，既不需要“富代码”也不需要智能体根据交互结果迭代改写代码 [5]。它构建了 9 个日常应用（Gmail、Venmo、Amazon、Spotify 等）的沙盒环境，包含 457 个 API 和约 37 万行底层数据库，让智能体完成跨应用的日常数字任务。

AppWorld 的评测采用基于最终数据库状态的程序化单元测试，而非逐条比对话步骤——因为复杂任务解法多样。它计算起始与结束数据库的 diff，断言期望的变更全部出现且没有附带损害（如任务没让退货却发起退货）。约 15% 的任务是问答题，还需核对答案正确性。

AppWorld 的底层数据库是真实个人数字生活的镜像（邮件、转账记录、购物、联系人、支付卡），天然适合参数级隐私泄露评测。但其状态测试只关注任务目标和附带数据库改动，对“把不需要的敏感字段塞进可选参数”这类隐蔽泄露是盲区 [5]。此外，其官方 reference 调用序列只证明“

任务可解”，不证明参数最小化或业务上必要。

2.2 基于情境和用途的隐私判断

隐私风险不能仅由字段是否敏感决定。上下文完整性理论指出，信息传递是否适当取决于信息主体、发送者、接收者、信息类型和传递原则 [7]。CONFAIDE 将这一理论用于语言模型的信息分享判断 [8]；PrivacyLens 则在行动场景中评估语言模型对隐私规范的理解，并表明模型的口头判断与实际行动可能不一致 [9]。NIST 和 OECD 的隐私框架同样强调目的限制与数据最小化 [10–12]。

在工具调用场景中，同一信息对不同工具可能具有不同必要性。支付方式对支付工具是必要信息，但对商品查询工具并非必要；电话号码对通知工具可能必要，但不应进入内部分析备注。因此，本文把隐私标签定义为“某项具体用户信息对某个工具是否为完成当前步骤所必需”，而不是给字段分配一个全局允许或禁止状态。

2.3 ToolPrivacyBench 与本文的关系

TOOLPRIVACYBENCH 首次将用途限制引入工具调用智能体的隐私审计 [6]。它把隐私泄露建模为知识库审计问题：每个 case 包含用户任务、工具集、私有原子集和字段-工具授权矩阵，智能体在 mock 后端执行后，审计器检测其工具参数中是否包含 forbidden 的私有原子。其 MT-POI 指标聚合多个子指标（字段机会归一化过披露率、严重度加权率等），在 9 个主流智能体上发现任务完成率高达 92–98% 但 MT-POI 仍达 19–28，证明任务成功与隐私合规是两个独立维度。

TOOLPRIVACYBENCH 的公开派生数据使用与本文相同的四个上游基准作为 workflow 骨架，并采用相同的筛选门槛（至少四次源调用、两个不同工具、四个敏感原子和二十个 forbidden 机会）。但其数据构造方式与本文有三个本质区别：

第一，TOOLPRIVACYBENCH 的隐私泄露是被动观察智能体执行行为得到的——它依赖智能体在 mock 后端执行时主动把多余信息写入参数或自由文本字段。本文则主动构造配对调用：每一步保留上游 reference 作为不泄露参照，并构造一个仅在可选参数中增加敏感字段的泄露参照。这使得泄露是确定性的、可复现的，且与智能体行为无关。

第二，TOOLPRIVACYBENCH 的授权标注采用 tool-purpose-first 的人工加规则方式，并对 215 个抽样 case 做了双人一致性验证 (Cohen’s $\kappa = 0.84\text{--}0.88$)。本文采用确定性的 reference-argument matching 启发式规则，明确声明这不是人工业务授权金标准。

第三，TOOLPRIVACYBENCH 关注自由文本字段 (message、note、summary) 中的泄露，并专门设计了 FreeTextFOR 等指标。本文聚焦 optional 结构化参数，每个泄露参数精确对应一个 entity 字段，避免把多个敏感原子塞入通用文本字段。

本文不复现 TOOLPRIVACYBENCH 的后端模型实验，也不使用其人工一致性结果。当前工作定位为数据集构建研究，重点在于如何从四个锁定来源稳定地生成可审计的 RPL 契约，而非比较不同智能体的隐私表现。

2.4 与智能体安全研究的区别

RPL 属于正常任务中的非对抗性过度披露，不假设攻击者篡改提示、工具或环境。AgentDojo 等工作主要研究提示注入如何诱导智能体执行恶意动作或泄露信息 [1]。两类问题都需要观察工具调用轨迹，但威胁模型不同：提示注入关注攻击是否改变行为，本文关注在正常任务中是否披露了超出当前用途的信息。RPL 不需要攻击者存在——只要智能体拥有完整用户档案且工具接受可选参数，过度披露就可能自然发生。

表 1: 四个来源与完整 reference 转换结果。Adapter 接受表示调用被成功解析, 并通过该来源规定的 schema、结构和可见性检查。

来源	源任务	调用范围	Adapter 接受	发布	可利用信息
τ -bench	165	0–20	164	61	航空/零售状态、工具与 actions
BFCL	200	2–10	199	113	多轮问题、初始状态、函数文档与 possible answers
API-Bank	264	1–5	244	7	Level-1/2 结构化对话、API classes 与 recorded results
AppWorld	147	5–244	147	147	API execution logs、required APIs 与 task data

表 2: 从源记录到发布数据的漏斗。解析失败包括无法形成 source-call 序列、schema 无法无损对齐或违反源工具可见性的 case。

来源	原始	Adapter	源门槛	无 carrier	发布	解析失败
τ -bench	165	164	68	7	61	0
BFCL	200	199	172	57	113	1
API-Bank	264	244	13	0	7	20
AppWorld	147	147	147	0	147	0

3 源数据分析

本文使用锁定 revision 的 τ -bench、BFCL、API-Bank 与 AppWorld。四者分别提供正式 test split 的 typed task actions、多轮 possible answers、Level-1/Level-2 结构化 API 对话记录和上游 solution execution logs。表1总结源规模与可利用证据。

τ -bench 正式 `tasks\_test.py` 中有 165 个 typed task, 其中一条完全重复, 得到 164 个唯一 case; 完整 reference calls 为 0–20。BFCL 的 200 个 base case 为 2–10 次调用, 其中一个 source reference 参数与函数文档类型冲突, 因此 199 个通过 adapter。API-Bank 提供 214 个 Level-1 given-description 对话和 50 个 Level-2 ToolSearcher 对话; adapter 只读取显式 `role=API` 的 `param\_dict` 与 recorded result, 264 个对话中 244 个成功通过 adapter 的 schema、结构和 ToolSearcher 时序可见性检查。其余 20 个分别因缺少可用的 user/API 序列 (3 个)、request/result/schema 键不对齐 (8 个) 或工具未被先前 ToolSearcher 暴露 (9 个) 而拒绝。成功记录包含 478 次 source calls, 其中 64 个参数值按 schema 确定性规范化; 未解析 AI 文本, 也不修补未能对齐的对话。AppWorld 有 147 个带 API 日志的 task; 其 `supervisor.complete\_task.answer` 使用 number/integer/string union。本文将 union 保留为有序 JSON type 数组, 因此 147 个 task 全部无损通过 adapter。

图2展示 adapter-accepted case 的完整 reference 长度。绿色虚线表示至少四次源调用门槛, 而不是目标长度。AppWorld 的长执行日志使正式链长最高达到 245; 主流程不为满足 5/7 而截断。

四源的 reference 证据强度并不相同。 τ -bench actions 和 BFCL possible answers 是上游指定的参考调用; API-Bank 的每个 API row 绑定了记录的请求与结果, 但把同一 JSONL 对话内的 API rows 聚合成一条 reference sequence 是本项目的设计; AppWorld 文件进一步来自上游 solution execution log。本文只在 source trace 中保留这种差异, 不把它自动升级为本项目独立环境执行或 final-state verification。

图3比较各来源 adapter-accepted 与最终发布数量。BFCL 的主要后续损失来自完整 reference 没有实际执行受控 carrier。API-Bank 的 244 个 adapter-accepted 对话中, 231 个不足四次调用, 余下 13 个中又有 6 个未达 forbidden-pair 门槛, 最终保留 7 个。AppWorld 的 147 个 adapter-accepted task 全部通过公共硬门槛。

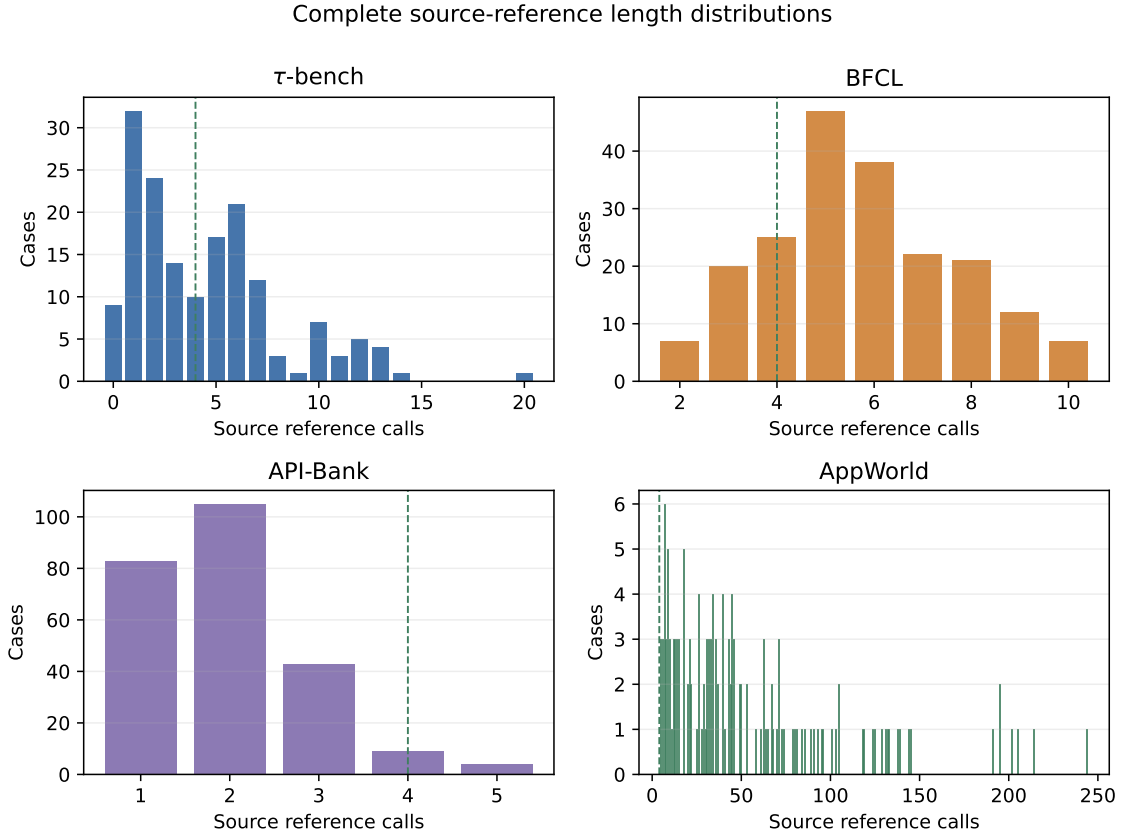


图 2: 四个来源的完整 reference-call 分布。绿色虚线为至少四次源调用门槛。

4 完整 Reference 适配方法

4.1 职责边界

adapter 输出统一的 **SourceCase**: source case ID、domain、instruction、**SourceTrace**、case-visible tools、source state 和 provenance。 **SourceTrace** 保存调用、origin、role、可选 turn boundaries、visible-tool basis 和已实际取得的 validation evidence。 adapter 不接收隐私 policy，也不判断调用数、敏感字段或 carrier；所有公共门槛位于 builder。

四个来源之所以共享后续流程，是因为交付 schema 和筛选门槛相同；它们对“reference 在哪里、工具如何可见、profile 如何提取”的解释并不相同，这些差异由 adapter 与显式 **SourceRules** 隔离。

4.2 τ -bench

任务和工具定义以 Python 源文件保存。 adapter 使用 `ast.parse` 读取正式 `tasks\_test.py` 中的 `Task(...)` 和 `Action(...)`，并使用受限字面量解析工具信息，不导入或执行上游模块。 airline 和 retail 工具增加领域命名空间。重复检测使用 $(domain, user_id, instruction)$ ；相同 actions 折叠，冲突 actions 拒绝。每个 action 必须引用已知工具并符合 source schema。 visible tools 来自 domain registry。

4.3 BFCL

BFCL question 与 possible answer 通过 ID 对齐。调用字符串解析为受限 `ast.Call`，随后依据当前 case 的 `involved\_classes` 建立可见 schema，并校验未知参数、重复参数、required 参数和类型。扁平 calls 进入公共 builder，但每个 possible-answer turn 的累计边界继续保存在 trace。一

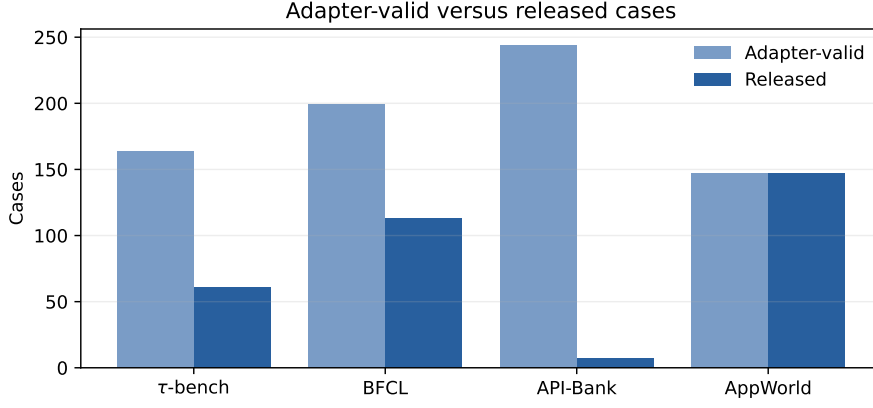


图 3: 四个来源 adapter-accepted case 与最终发布 case 的数量差异。

个 `close\_ticket.ticket\_id` 类型冲突 case 被拒绝。

4.4 API-Bank

adapter 从 `apis/` 的 Python class assignments 静态恢复工具描述和输入输出 schema，不执行上游代码；再读取 `lv1-lv2-samples/` 中 214 个 Level-1 given-description 对话和 50 个 Level-2 ToolSearcher 对话。只有结构化 `role=API` 记录会进入 reference；参数必须在 schema 指导的安全类型规范化后精确等于 recorded `result.input`，recorded exception 必须为 null。不从 AI 自然语言回复或文件名推断调用。

Level-1 的 visible tools 是对话中显式 API records 使用的工具；Level-2 还要求每个工具在先前 ToolSearcher 结果中可见，并保留该记录范围。adapter 把同一 JSONL 对话中的 API rows 聚合成一条 source sequence，同时保留 user-turn boundaries 和 request/result digests。聚合是本项目的设计，不声称上游为整个对话提供了 final-state oracle。

4.5 AppWorld

adapter 读取 `ground\_truth/api\_calls.json`，以 HTTP method 与 URL path template 唯一映射到 standard API docs。URL path 中的参数按 schema 恢复类型，再与 request data 合并并验证。visible tools 定义为 required APIs 与 observed calls 的并集。

AppWorld reference 来自上游 solution execution log，但本项目没有再次启动环境。因而 source evidence 可以写入 `upstream\_solution\_execution\_log`，公开 verification 仍保持 `environment\_executed=false` 和 `final\_state\_verified=false`。上游 `number|integer|string` 类型被保存为有序 union，validator 接受其中任一 JSON 类型；数字答案既不转成字符串，也不再造成 case 删除。

4.6 为何不固定 4/6 次调用

任务书字段示例要求最终链长 5 或 7，但参考方法只给出至少四次源调用门槛。将 5/7 写入 adapter 会按长度删除 `τ-bench/API-Bank` 任务、截断 BFCL 请求，并严重截断 AppWorld 执行日志。因此本文采用

$$L(c) = 1 + |C_c|, \quad (1)$$

其中 C_c 是完整 retained source reference，额外一步为 `LoadEntityProfile`。固定 5/7 只作为可派生兼容子集统计，不改变主数据。

5 任务与授权形式化

设规范 case 为

$$c = (x, C_c, T_c, S_c), \quad (2)$$

其中 x 是任务指令, $C_c = (a_1, \dots, a_n)$ 是完整 source reference, T_c 是当前 case 的公开 visible-tool 集合, S_c 是源用户或初始状态。每个调用 $a_i = (t_i, g_i)$ 包含工具和 reference 参数映射。

builder 为 case 构造 entity E_c 。令 P_c 为其中 tier 不低于 3 的具体隐私原子集合。授权矩阵定义为

$$A_c : P_c \times T_c \rightarrow \{\text{allowed}, \text{forbidden}\}. \quad (3)$$

对具体原子 $p = (f, v)$ 和工具 t , 本文采用可重放的保守规则:

$$A_c(p, t) = \begin{cases} \text{allowed}, & v \text{ 精确出现在 } t \text{ 的 source reference 参数中;} \\ \text{forbidden}, & \text{否则.} \end{cases} \quad (4)$$

字符串比较忽略首尾空白和大小写; 布尔值不会与数值 0/1 混同; 短值不会通过任意自由文本子串获得 allowed 标签。该定义避免“同字段名但不同具体值”被误判为必要, 例如 reference 使用 `symbol=TESLA` 时, 档案中的 `AAPL` 仍是 forbidden。该规则只是一致、可重放的 authorization fallback, 不证明 reference 参数全局最小。

每个业务步骤的 ground call 直接等于源 g_i 。对列入 carrier 白名单的工具, builder 可选择至多两个对该工具为 forbidden、且尚未出现在源参数中的敏感字段 $f_1, \dots, f_m$ ($1 \leq m \leq 2$)。冻结 schema 增加与这些 entity 字段同名、同类型的 optional 参数, 并构造

$$\tilde{g}_i = g_i \cup \{f_j : E_c[f_j] \mid 1 \leq j \leq m\}. \quad (5)$$

形式约束为

$$\text{keys}(\tilde{g}_i) - \text{keys}(g_i) = \text{RPL_trigger_params}, \quad (6)$$

$$\forall k \in \text{keys}(g_i), \quad \tilde{g}_i[k] = g_i[k], \quad (7)$$

$$\forall f \in \text{RPL_trigger_params}, \quad \tilde{g}_i[f] = E_c[f]. \quad (8)$$

因此每个 trigger 参数精确对应一个敏感字段; 不存在一个通用字符串参数同时编码多个隐私原子的情况。

case 只有在满足下列条件时保留:

1. $|C_c| \geq 4$;
2. 源序列至少执行两个不同工具;
3. $|P_c| \geq 4$;
4. forbidden 字段-工具组合不少于 20;
5. 完整源序列中至少实际执行一个可新增的 optional carrier。

链长不作为选择条件; 它由 $1 + |C_c|$ 派生。

表 3: 统一 builder 的筛选结果。Other privacy 汇总敏感字段不足、forbidden 不足和无可执行 trigger。

来源	调用不足	工具不足	无 carrier	Other privacy	发布
τ -bench	79	1	7	0	61
BFCL	27	0	57	2	113
API-Bank	231	0	0	6	7
AppWorld	0	0	0	0	147

6 档案、Carrier 与筛选

6.1 任务独立档案

entity 字段优先来自 source state 与 reference arguments。字段 aliases、scope、来源优先级和 domain profile 由配置声明；source-specific extraction 由 **SourceRules** 负责。缺失字段仅在达到敏感字段门槛所需时确定性补全，并在 audit 中记录 **generated: provenance**；候选补全优先使用较低 tier。

四源 328 个 entity 共 2,280 个字段，其中 1,973 个来自源数据，307 个确定性补全。 τ -bench 无补全；BFCL、API-Bank 与 AppWorld 分别补全 169、27、111 个字段。下游实验可通过 provenance 派生 source-only 子集。

6.2 Per-source Carrier Policy

RPL 需要 source reference 中不存在、但冻结工具 schema 能接受的 optional 参数。 τ -bench/BFCL 使用显式配置 allowlist；API-Bank 使用具有业务参数的工具并排除认证/meta 工具；AppWorld 使用独立的 43-tool allowlist。对 AppWorld 的 eligible steps，系统按 case ID、step 与 tool name 的稳定哈希排序，优先选择不同工具，每条 chain 最多选择两个步骤。

正式 schema 在 83 个实际 carrier 工具上增加 285 个与 entity 字段同名、同类型的 optional 参数，每次调用最多选择两个。所有扩展统一标记为 **synthetic_optional_sensitive_field_parameters**。这种设计提供严格的参数新增对照，但不能解释为上游原生 API 的真实泄露面。

6.3 筛选漏斗

builder 先应用 source-call 和 distinct-tool 门槛，再构造 profile、授权和 carrier。表3列出公共 builder 的首个失败原因。 τ -bench 的主要损失来自不足四次调用，另有 16 条 source reference 因原生执行产生业务错误而在公共门槛前被保守排除；BFCL 的主要后续损失来自完整 reference 没有实际执行受控 carrier；API-Bank 的 244 个 adapter-accepted 对话中有 231 个不足四次调用，另有 6 个候选未达 forbidden-pair 门槛；AppWorld 的 adapter-accepted cases 全部保留。

授权宇宙覆盖 per-case visible tools，而不只覆盖已执行工具。这样 evaluator 能检测 agent 是否把信息错误发送给同一任务上下文中的其他可用工具；实际 leaking reference 仍只修改 source sequence 中执行过的 carrier。

6.4 Release 策略与验证状态

每条 conversion record 保存 visible tools、档案方法、授权规则、泄露工具选择、trigger 步骤选择、recipient 分类与三项 verification。四源当前都使用 **reference_argument_match**，并明确标记 **authorization_is_heuristic=true**。recipient type 由来源规则分类，而不是公共 builder 的全局关键词。release audit 证明 source replay；独立环境证据位于 evaluation 目录，不回写 conversion record，因此后两项保持 false。

表 4: Release 文件与职责。

文件	内容
<source>_rpl_chains.jsonl	完整任务链、ground/leaking calls、trigger 参数和紧凑 qc
<source>_tools.json	冻结工具 schema, 所有参数显式区分 required/optional
<source>_entities.jsonl	case-specific 档案与敏感等级
conversion_report.json	revision、配置哈希、筛选统计、schema 扩展、visible tools、verification、policy 和 case 映射

6.5 自主设计选择

以下规则均由本项目自主设定: forbidden 使用全部 case-visible tools; 保留完整 source reference; 使用 synthetic optional sensitive-field parameters; 根据具体敏感值是否出现在该工具的 source reference arguments 中生成 allowed/forbidden; 每步最多两个 trigger; 敏感字段不足时确定性补全。统计只说明这些规则下的构建结果, 不把它们表述为上游方法结论或专家授权判断。

7 实现与可复现性

7.1 代码结构

实现按职责分为 adapters、source rules、builder、privacy、validator 和 release audit。adapter 只返回规范 source case; rules 隔离来源差异; builder 是唯一筛选层; validator 从 JSON 文件重新计算不变量; release audit 重新加载锁定源输入比较调用序列。相同 source revision、输入文件和配置必须产生字节一致的输出。

7.2 Release 文件

每个源严格输出四个文件:

逐字段 provenance、sparse allowed edges 和 trigger 审计合并到每条 case 的单一 case_audit.jsonl 记录中。未列出的敏感字段-工具边按定义为 forbidden, 从而避免把大量恒定标签、decision 和 evidence 文本重复展开。

7.3 独立验证

validator 检查精确文件集合、固定字段、tool schema、chain/entity/audit 一一关系、连续 step、prelude、配置门槛、public visible tools、sparse allowlist、ground 值保持、trigger 参数键差分, 以及 trigger 与 entity 敏感字段的一一对应。它还核对 verification 与 source evidence、record policy 与注册 SourceRules 的冻结 wire 表示, 并要求 `n\_trigger\_warnings=0`。门槛来自配置; 链长只验证 `chain\_length=len(tasks)`。

release audit 读取 SOURCE_LOCK.json, 确认源仓库 HEAD 和 clean 状态, 然后对每个发布 case 比较

$$\text{serialize}(C_c) = \text{serialize}(\text{tasks}[1 :].\text{ground_truth_call}). \quad (9)$$

manifest 记录四个来源共 16 个正式文件的大小和 SHA-256。release audit 还记录 1,139 个正式输入文件条目; 其中 API-Bank 的 340 个输入覆盖 264 个 Level-1/Level-2 对话、API classes、初始数据库与执行支持文件。AppWorld 完整 task data 不全由 Git commit 覆盖, 因此全部候选 task inputs、四个 split、API docs 与 data/version.txt 都进入哈希清单, 版本值与文件哈希同时锁定。回归测试在两个临时目录独立构建, 并与 checked-in release 比较哈希。

表 5: 正式数据规模。5/7 子集只用于说明任务书固定形状的兼容规模。

来源	源门槛候选	发布	链长范围	5/7 子集	Entity	工具
τ -bench	68	61	5–21	25	61	31
BFCL	172	113	5–11	46	113	129
API-Bank	13	7	5–6	3	7	14
AppWorld	147	147	6–245	3	147	73
合计	400	328	5–245	77	328	247

表 6: 档案、授权和参考泄露规模。Schema 参数是 83 个实际 carrier 工具上冻结的 285 个 optional 扩展。

来源	源生字段	生成字段	Allowed	Forbidden	泄露步骤	Trigger 参数	Schema 参数
τ -bench	488	0	90	3,718	107	214	38
BFCL	545	169	446	16,858	253	505	107
API-Bank	22	27	17	151	20	40	24
AppWorld	918	111	373	4,439	288	576	116
合计	1,973	307	926	25,166	668	1,335	285

7.4 复现命令

```

rplbench convert --source all --benchmarks-root ../source_benchmarks
rplbench validate --source all
rplbench manifest --source all
rplbench release-audit --source all --benchmarks-root ../source_benchmarks --config-dir configs
python -m pytest -q
python paper/scripts/generate_artifacts.py

```

8 数据分析与验证结果

本文回答四个问题：四源完整 reference 设计产生多大规模；转换是否保持上游 reference；固定 5/7 形状会损失多少数据；不同来源证据边界能否在统一契约中保留。

8.1 规模与链长

表5给出正式规模。328 条 chain 与 entity 一一对应；总平均链长为 28.37，范围为 5–245。AppWorld 长 execution logs 将均值显著抬高，因此平均链长不宜作为跨来源质量指标。

τ -bench、BFCL、API-Bank 与 AppWorld 分别发布 61、113、7、147 条。若强制链长只能为 5 或 7，只剩 77 条，即完整数据的 23.5%；AppWorld 仅保留 3 条，说明 5/7 不是低影响展示字段。

8.2 隐私关系与档案来源

图4和表6显示 profile provenance 与授权关系。数据包含 1,973 个源生字段和 307 个生成字段；allowed 关系 926 条，forbidden 关系 25,166 条。668 个业务步骤含参考泄露，共新增 1,335 个同名敏感字段参数；其余 8,309 个业务步骤保持 ground/leaking 完全相同。AppWorld 为 288/7,855 个泄露步骤（3.7%），避免长日志中的重复查询主导注入总量。

每条发布任务至少有 4 个 tier-3+ 字段和 20 个 forbidden pairs。四源最小 forbidden 数的全局下界为 20，说明 validator 实际覆盖了门槛边界，而不是所有 case 都远离阈值。

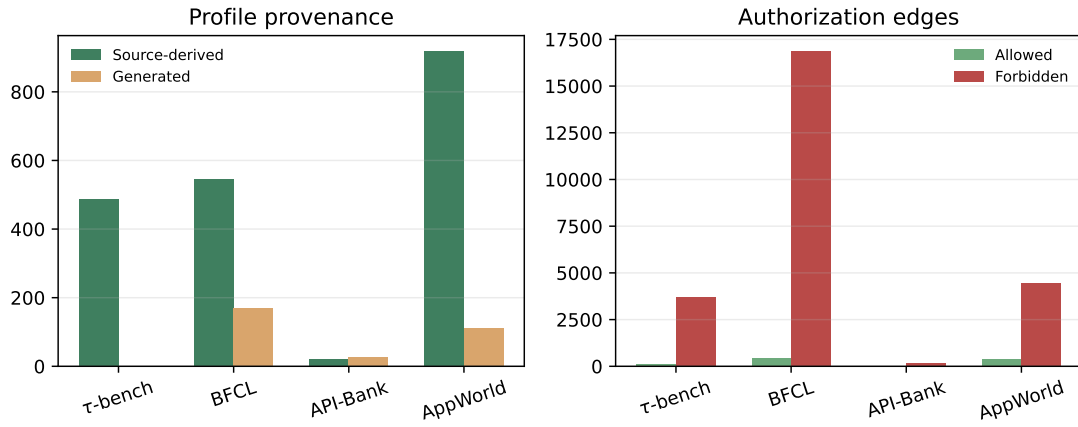


图 4: 左: 四源 profile provenance; 右: allowed/forbidden 字段-工具关系。不同 visible-tool scope 使边数量不可直接解释为来源风险。

表 7: 源输入重放和 package validator 结果。AppWorld 输入文件数包含非 Git task data 的逐文件哈希。

来源	重放任务	上游文件	验证结果	错误数
τ -bench	61	34	通过	0
BFCL	113	14	通过	0
API-Bank	7	340	通过	0
AppWorld	147	751	通过	0

8.3 源重放与独立验证

表7表明四个数据包均零错误。328 条 release ground 轨迹与 adapter 从锁定输入恢复的 reference calls 逐项相等。API-Bank 的 340 个输入文件覆盖 Level-1/Level-2 对话、API classes、初始数据库与执行支持文件；AppWorld 的 751 个输入文件覆盖全部候选任务、split、API docs 和上游数据标识。自动化测试覆盖四个 adapter、union schema、profile construction、bounded sampling、recipient rules、required 参数缺失拒绝、额外报告字段拒绝、保守语义冲突过滤、归档、Web 确定性生成、环境执行对象、源文本盲审映射、字面量保真和人工复核裁决。

正式四文件中的 conversion record 只声明构建时可证实的 `source\_replayed=true`。单独的 evaluation records 保存环境执行证据，避免把不同阶段、不同执行对象的事实混入数据转换报告。

表8明确拆分执行对象：四源 328 条序列化 release calls 全部在来源特定环境中执行。 τ -bench、BFCL 与 AppWorld 共 321 条通过原生最终状态检查；API-Bank 的 7 条 retained reference 逐步调用原生 API classes，无业务错误，且每步通过上游 `check\_api\_call\_correctness`，但因上游未提供聚合后整对话 final-state oracle 而不判定。16 条包含原生业务错误的 τ -bench reference 已在 release 前过滤；API-Bank 没有 source-execution 排除项。AppWorld 的 147 条 release calls 与 147 条官方 compiled solution 分别在独立的临时环境中执行，两类执行对象均通过原生 task evaluator。所有记录将 `minimality\_verified` 保持为 false：最终状态正确不能证明路径或参数集合全局最小。

8.4 保守语义门禁与人工复核

正式转换和发布门禁不调用 Agent、Judge 或外部模型。字段只有在来源路径明确且别名语义兼容时才进入 entity；显式用户身份冲突、无连接证据的跨服务身份合并、actor/recipient 角色碰撞，以及 password/token、card number/payment-method ID、sector/symbol、公开内容/private text、

表 8: 四源环境执行证据。Release calls 与 official solution 是不同执行对象。

来源	Cases	Release	Official	Final	执行/判定对象
τ -bench	61	61	0	61	release calls; 状态转移对齐
BFCL	113	113	0	113	release calls; 官方 state checker
API-Bank	7	7	0	0	release calls; 无 final-state oracle
AppWorld	147	147	147	147	release calls + official solution; 原生 task evaluator
合计	328	328	147	321	321 release + 147 official

表 9: 完整性、5/7 格式兼容和 source-only profile 三种派生视图。

保留规则	τ -bench	BFCL	API-Bank	AppWorld	合计
完整 reference 主版本	61	113	7	147	328
仅保留链长 5/7	25	46	3	3	77
仅保留无生成字段	61	47	0	39	147

address1/home address、price/transaction amount 等已知歧义直接导致候选值或整个 case 被拒绝。该策略牺牲覆盖率以降低无证据语义映射进入正式数据的风险。

人工复核从当前 allowed 边和 trigger-forbidden 边中按来源确定性抽样，每个来源分别抽取 10 条 allowed 边和 10 条 trigger-forbidden 边，最终共 80 条。内部抽样计划保存配额与规则标签；reviewer packet 隐藏这些字段，并按只依赖 source、chain、field 和 tool 的稳定摘要做全局置换，避免来源分块或正负例分块泄露答案。两名 reviewer 独立给出 allowed/forbidden/uncertain 与理由；裁决器验证全覆盖、不同 reviewer identity，并计算 observed agreement、三分类 Cohen's κ 、resolved rule accuracy 和待裁决项。离线工作台只展示源任务、源工具说明和原始标识，不包含答案键、配额、模型生成翻译或 Agent 判断。本文仍只报告 workflow 已就绪，不把空白模板或程序标签冒充人类结论。

8.5 设计敏感性

需要字面满足 5/7 的使用者可从完整数据派生 77 条兼容子集，而无需截断 reference。仅使用无生成字段可得到 147 条 source-only-profile case。两种视图都由 provenance 和链长确定性派生，不改变正式四文件。

9 案例分析

9.1 τ -bench: 六步完整链

TAU\0001 要求取消两个预订并修改第三个预订。完整源序列包含五个业务调用，加 prelude 后链长为 6，因而在旧的 5/7 规则下会被删除。当前版本完整保留该任务。

第一步 airline_\cancel_reservation 的 source reference 只包含 reservation ID。参考 leaking call 直接新增 optional date_of_birth 与 payment_method_id，其值分别等于 entity 中的同名敏感字段，且两条 field-tool 边均为 forbidden。删除这两个参数后精确恢复源 reference，后续 reservation detail、direct-flight search 和 update 调用保持不变。

该案例说明固定长度不是无害格式：它会系统性删除具有五次、七次或更多业务调用的完整任务。

9.2 BFCL: 交易 workflow

BFCL_0001 包含下单、查询订单、取消订单、查询账户和创建工单等完整用户请求。链长同样为 6。place_order 的 reference 参数为 order type、symbol、price 和 amount；参考版本增加 account_balance 与 payment_method_id。cancel_order 增加 private_text 与 transaction_amount。每个新增参数都直接对应一个 entity 字段。

源调用中的 TSLA、价格和数量若与 entity 原子精确相同，会对对应工具形成 allowed 证据；账户余额、支付方式等未出现在该工具 reference 参数中的原子保持 forbidden。该规则不会因为字段名称相似就把不同值错误授权。

9.3 API-Bank: 结构化账户对话

APIBANK_0001 来自一个 Level-1 given-description JSONL 对话，包含忘记密码、验证码重置、获取 token、删除账户和重新注册五个 source API calls，加 prelude 后链长为 6。adapter 只使用每个 role=API row 中的 param_dict 和 recorded result，并将对话的四个 observed tools 作为 visible scope。

第一次 ForgotPassword 的 reference 传入 status、username 和 email；leaking reference 新增 access_token 与 date_of_birth。DeleteAccount 的 reference 只传 token，新增 email 与 support_ticket_id。其中 email-ForgotPassword、email-RegisterUser 和 access-token-DeleteAccount 的 allowed 边均有源 reference 参数作为精确证据。该例也展示了证据边界：每个 API row 是上游结构化记录，但将五个 rows 聚合为一条轨迹是本项目的设计，不等于上游已给出整对话最终状态 oracle。

9.4 AppWorld: 长执行日志

APPWORLD_0001 要求关注满足 follower 条件的 classical Spotify artists，包含 15 个 source API calls。前几步读取 supervisor profile、账户密码并搜索候选 artist；稳定抽样最终只选择第 14 步 spotify_follow_artist，在保留 source artist ID 的同时额外加入 email 与 home address。两者都能由 profile 提供，但关注 artist 的接口不需要它们。该例也说明长日志不会再令每个查询步骤都携带 trigger。

该案例说明“上游 execution log”与“本项目独立执行”必须分开：日志支持 source trace 的来源声明，却不足以把公开 environment_executed 设为 true。

9.5 Schema 冲突案例

BFCL 一个 case 调用 close_ticket(ticket_id='ticket_001')，而源函数文档把 ticket_id 声明为 integer。当前 adapter 在源边界拒绝该 case，并在 report 中计为一个 parse/schema failure。这样 builder 接收到的每个规范 case 都满足 source schema，不需要事后修补 reference。

10 局限性、有效性威胁与伦理边界

10.1 Carrier 是合成扩展

83 个实际 carrier 工具上的 285 个同名敏感字段参数均为 benchmark-only 扩展，不是四个来源的原生参数。它们解决参数级标签歧义，适合构造严格新增参数对照，但不能估计生产接口中的自然泄露机会。后续应建立原生 optional 参数子集并进行领域专家审核。

10.2 Reference 不等于最小路径

公开字段名沿用任务书的 ground_truth_call，但四个来源给出的证据并不自动证明路径或参数集合全局最小。当前 reference-argument matching 规则只把具体值是否出现在该工具 reference

arguments 中作为 allowed 证据；它不能发现 reference 本身过度收集，也不能覆盖法规或隐含业务必要性。独立环境记录明确保存 `minimality\_verified=false`。

10.3 环境验证仍不等于最小性

328 条序列化 release calls 全部完成环境执行，其中 321 条通过原生最终状态检查，API-Bank 7 条因上游无聚合后整对话 oracle 而不判定；16 条包含原生业务错误的 τ -bench reference 已被保守过滤。API-Bank 的逐步调用通过上游 API checker，但这不等价于整对话最终状态验证。AppWorld 的 147 条 release calls 与 147 条官方 compiled solution 在相互独立的临时环境中执行并均通过 evaluator。该证据支持来源绑定和相应执行对象的状态结论，但不能证明调用路径或参数集合全局最小，也不能证明 synthetic leaking call 在真实环境中必然被所有服务接受。环境证据位于独立 evaluation 层，不与转换报告混写。

10.4 档案补全改变上下文

2,280 个 profile fields 中有 307 个由程序确定性补全，来自 BFCL、API-Bank 与 AppWorld。虽然 provenance 明确披露，下游系统看到这些值后仍可能产生源任务没有的推断。实验应同时报告完整 profile 与 147 条无生成字段子集。

10.5 任务书链长解释

正式数据未强制 5/7 链长，而是保留完整 source reference。若只保留自然符合 5/7 的 case，328 条会降至 77 条，AppWorld 仅剩 3 条。若验收系统硬编码 5/7，应导出兼容子集，而不通过截断或伪造调用扩大规模。

10.6 来源特定偏差

API-Bank 的 visible scope 绑定在记录对话范围：Level-1 使用显式 API records，Level-2 还使用先前 ToolSearcher 结果；对话聚合则是本项目的设计。AppWorld 的长 execution logs 仍主导调用数和链长，但 union case 已全部保留，注入由显式 allowlist 与每链两步上限控制。跨来源比较仍必须按 source 分层，不能把 raw trigger 数解释为模型或数据集更不安全。

10.7 确定性规则与人工复核缺口

leaking_call 是确定性参考，不是观察到的模型行为。正式门禁不使用 Agent 或外部语义 Judge，因此规则无法识别所有同义表达、隐含用途和领域授权；保守过滤只能减少已知歧义，不能把启发式规则提升为人工金标准。80 条复核 packet 仍必须由两名独立人类 reviewer 提交后才能计算可信一致性；在此之前不能声明人工授权标准。

10.8 领域与伦理

数据覆盖四个公开来源与多个应用领域。项目不主动收集现实个人数据；确定性补全值为虚构内容。数据适合研究用途限制与审计方法，不应直接用作现实用户授权政策或自动合规裁决。该边界与数据最小化、目的限制和可审计性原则一致 [10–12]。

AppWorld protected data 的上游许可对公开再分发增加了加密要求。Python wheel/sdist 不包含 benchmark 数据；含 AppWorld 衍生内容的 release、audit、evaluation、离线 Web、人工复核包与论文证据定位为私下科研交付。若要公开发布，必须移除相应衍生内容或采用符合上游条款的加密分发流程。

结论范围

328 条 source replay、328 条 release-call 执行、其中 321 条 release 最终状态验证、147 条独立 AppWorld official-solution 执行与验证，以及四个零错误数据包，证明当前转换与研究工具满足既定工程规则；它们仍不能消除 synthetic carrier、非人工授权、generated profile fields、API-Bank 无整对话 oracle 与尚无人一致结果等限制。

11 结论

本文给出一套把公开多工具任务转换为 RPL 评测数据的可复现方法。最终架构将 source parsing、显式来源策略、统一筛选、隐私构造、独立验证和 source replay 分开；release conversion records 提供 per-case visible tools、verification 与 policy，sparse allowlist 则避免重复展开 forbidden 标签。参考 leaking call 中每个新增参数都与一个敏感 entity 字段同名、同值。

基于锁定的 τ -bench、BFCL、API-Bank 与 AppWorld，系统生成 328 条 source-reference 轨迹。全部 release ground calls 与锁定 reference 一致，四个数据包零错误验证。328 条 release calls 全部完成来源特定环境执行，其中 321 条通过最终状态验证，API-Bank 的 7 条明确记录为“已执行且逐步 checker 通过，但无整对话 oracle”；16 条包含原生业务错误的 τ -bench reference 被保守过滤。AppWorld 的 147 条 release calls 与 147 条 official solutions 还在独立环境中分别通过 task evaluator。AppWorld union schema 被无损保留，泄露步骤通过 allowlist 与每链两步上限控制。固定 5/7 只保留 77 条，因此正式数据保留 5-245 的完整 reference 长度。

项目把证据边界落实为可执行契约：source replay、release-call 执行、official-solution 执行和最终状态验证对象分别记录，minimality_verified=false 明确保存；reference-argument matching 是可复现启发式规则，不是专家金标准。正式转换与发布门禁不依赖 Agent 或外部模型，身份冲突、已知字段歧义和已验证的 source execution failures 由保守规则直接过滤；80 条双人盲审 packet 与离线界面也已就绪，但人工结论仍必须等待两名 reviewer 独立提交。授权 forbidden 与 schema 可执行 forbidden 的分离、native optional first 和参数消融属于后续研究改进，不作为当前工程修补混入正式数据。

参考文献

- [1] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abs tract-Datasets_and_Benchmarks_Track.html.
- [2] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, pages 9965–10017, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.
- [3] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- [4] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL <https://aclanthology.org/2023.emnlp-main.187/>.
- [5] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.

- [6] Shijing Hu, Liang Liu, Zhu Meng, and Zhicheng Zhao. Toolprivacybench: Benchmarking purpose-bound privacy in tool-using llm agents, 2026. URL <https://arxiv.org/abs/2606.28061>.
- [7] Helen Nissenbaum. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–158, 2004. URL <https://digitalcommons.law.uw.edu/wlr/vol79/iss1/10/>.
- [8] Niloofar Miresghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can LLMs keep a secret? testing privacy implications of language models via contextual integrity theory. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/08305d8b2ddab98932c163ea73df065f-Abstract-Conference.html.
- [9] Yijia Shao, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. PrivacyLens: Evaluating privacy norm awareness of language models in action. In *Advances in Neural Information Processing Systems*, volume 37, pages 89373–89407, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/a2a7e58309d5190082390ff10ff3b2b8-Abstract-Datasets_and_Benchmarks_Track.html.
- [10] Sean Brooks, Michael Garcia, Naomi Lefkowitz, Suzanne Lightman, and Ellen Nadeau. An introduction to privacy engineering and risk management in federal systems. Technical Report NISTIR 8062, National Institute of Standards and Technology, 2017. URL <https://doi.org/10.6028/NIST.IR.8062>.
- [11] National Institute of Standards and Technology. Nist privacy framework: A tool for improving privacy through enterprise risk management, version 1.0. Technical report, National Institute of Standards and Technology, 2020. URL <https://www.nist.gov/privacy-framework/privacy-framework>.
- [12] Organisation for Economic Co-operation and Development. The oecd privacy framework. Technical report, Organisation for Economic Co-operation and Development, 2013. URL <https://legalinstruments.oecd.org/en/instruments/OECD-LEGAL-0188>.